

Analysis: Candies

Candies: Analysis

To make our discussion easier, for each range (L, R) , let us define $SS(L, R)$ to be the sum of sweetness of the candies from index L to R (inclusive), and $SO(L, R)$ to be the number of odd candies from index L to R (inclusive). Our problem is equivalent to finding a range (L, R) such that:

- $SO(L, R) \leq O$.
- $SS(L, R) \leq D$.
- $SS(L, R)$ is maximized.

Small dataset

We will use the technique of **two pointers** to solve the Small dataset. For each value of L from 1 to N , we would like to find the value of R such that the value of $SS(L, R)$ is maximized while observing the constraints stated above. Since the candies have non-negative sweetness in the Small dataset, for every L , we simply have to find the rightmost index R that satisfies $SO(L, R) \leq O$ and $SS(L, R) \leq D$.

For any fixed L' , suppose we have identified that R' is the rightmost index that satisfies $SO(L', R') \leq O$ and $SS(L', R') \leq D$. For $L'+1$, note that $S(L'+1, R')$ necessarily satisfies $SO(L'+1, R') \leq O$ and $SS(L'+1, R') \leq D$. Thus, we do not need to iterate through the entire array to find the value of R that maximizes $SS(L'+1, R)$. We simply have to iterate from $R' + 1$ to N to find the largest value of R that still satisfies our constraints. This dramatically reduces the runtime of the algorithm, as we only iterate the array once to find the optimal value of R for each L (Hence the name two pointers — the pointers L and R only iterate over the array once). Once we obtain the optimal sweetness for each L , we can simply take the maximum and return as the answer. Total runtime of the solution is $O(N)$.

Large dataset

Since S_i may be negative in the Large dataset, for each L , we can no longer assume that the rightmost index that satisfies the constraints will give us the optimal value for $SS(L, R)$. However, we can still use the above sliding window technique to identify the rightmost index that still satisfies the oddness constraint. For any fixed L' , suppose we have identified that R_O' is the rightmost index that satisfies the oddness constraint. That is to say, we have $SO(L', R_O') \leq O$ and $SO(L', R_O' + 1) > O$. Now we need to find the index R in the range $[L', R_O']$ that maximizes the value of $SS(L', R)$, while observing the constraint $SS(L', R) \leq D$.

To do so, we shall precompute the prefix sum of the sweetness of the candies (i.e. the values of $SS(1, 1), SS(1, 2), \dots, SS(1, N)$). Notice that $SS(L, R) = SS(1, R) - SS(1, L - 1)$. Thus, finding the value of R that maximizes $S(L', R)$ is equivalent to finding the value of R that maximizes $SS(1, R) - SS(1, L' - 1)$, which is the same as finding **the value of R that maximizes the prefix sum $SS(1, R)$ while observing $SS(L', R) \leq D$** .

In order to identify the value of R quickly, we will need a data structure that supports adding integers, removing integers, and finding the largest integer smaller than a queried integer. A **balanced binary search tree** (e.g. C++ multiset) can perform each of these operations in logarithmic time. As we iterate over the value of L , we will add or remove the values of $SS(1, R)$

to the tree such that the tree only contains prefix sums for the indices in the range $[L, R_O]$. Then, we will query for the largest prefix sum such that $SS(1, R) \leq SS(1, L - 1) + D$. This allows us to compute the maximum sweetness for a subarray that starts from L , and we may then compute the overall maximum sweetness across all subarrays.

The time needed to preprocess all prefix sums and values of R_O is $O(N)$. Subsequently, each prefix sum may only be added or removed at most once from our data structure. In addition, we will only query the data structure once for each value of L . Using a balanced binary search tree, this gives us a total runtime of $O(N \log N)$.

Analysis: Funniest Word Search

Funniest Word Search: Analysis

Small dataset

As we approach the problem, we want to decide on what we can feasibly calculate. Since the upper bounds for **R** and **C** are reasonably low, we know that we can iterate through all combinations of $(i1, j1, i2, j2)$ where $(i1, j1)$ is the top left cell of a subgrid and $(i2, j2)$ is the bottom right cell; however, recalculating the total length of words found for every subgrid will still take too long. Note that since the length of each valid word is 1 in the Small dataset, the total length from finding a single instance of a word is 4. One solution is to calculate prefix sums $S(i, j)$ and then calculate $\text{totals}[i1, j1, i2, j2] = S(i2, j2) - S(i1 - 1, j2) - S(i2, j1 - 1) + S(i1 - 1, j1 - 1)$.

The prefix sum solution is simpler, but we will examine a different algorithm that extends more readily to the Large dataset. For every $i1$, create an array $\text{column_sum}[j]$ which, when we iterate through values for $i2$, will represent the total length of words found in the column bounded by $(i1, j, i2, j)$. For every $i2 \geq i1$, scan through row $i2$ to update $\text{column_sum}[j]$ by adding 8 if $(i2, j)$ is a word. After doing so, for every $j1$, start with $\text{totals}[i1, j1, i2, j1] = \text{column_sum}[j1]$, and for every $j2 > j1$, update $\text{totals}[i1, j1, i2, j2] = \text{totals}[i1, j1, i2, j2 - 1] + \text{column_sum}[j2]$. As we calculate $\text{totals}[]$, keep track of the best *fun size* and how many times it occurs. As a side-note, it is not necessary to store the entire $\text{totals}[]$ array, since we only use one previous calculation. This algorithm takes $O(R^2C^2)$ time.

Large dataset

Knowing that all words are of length 1 is very beneficial in the Small dataset for quickly updating our column sums. Luckily, with a bit of precomputation, updating can be just as easy for the Large dataset. As long as our precomputations are not slower than $O(R^2C^2)$ time, we should not have any problems.

Before running our algorithm, we can add the reverse of every word to our dictionary, so we only need to scan for words going from left to right (now referred to as going right) and top to bottom (going down).

We can observe that the length of a word limits where it can begin. Using this property, we can reduce the number of words to search for at each location. As a result, we want to create a lookup table $\text{word_lookup}[i]$ which has a list of words of length i at each respective index. Now let us use two more lookup tables $\text{right}[i, j, k]$ and $\text{down}[i, j, k]$ which will keep track of how many words start at cell (i, j) and have length $\leq k$ going right and down, respectively. We can populate the tables in the following manner: for every (i, j) start with length $k = 1$. While k is a valid length for a word going right starting at (i, j) with respect to the entire grid, check every word in $\text{word_lookup}[k]$ to see if it can be found going right starting at (i, j) . Record $\text{right}[i, j, k] = (k * \text{number of words found}) + \text{right}[i, j, k - 1]$. Increment k until it is not a valid length for a word starting at (i, j) . Repeat for words going down instead of going right.

Now, we can use an algorithm which is very similar to the one we used on the Small dataset. Let us just focus on words going right. We will still use $\text{totals}[i1, j1, i2, j2]$ for the same purpose. For every $i1$, have a table $\text{right_column_sums}[j, k]$ which, when we iterate through values for $i2$, will represent the total length of words going right with length $\leq k$ which start at the column bounded by $(i1, j, i2, j)$. For every $i2 \geq i1$, scan through row $i2$ and update $\text{right_column_sums}[j, k]$,

$k] = \text{right_column_sums}[j, k] + \text{right}[i2, j, k]$ for all valid values of k (remember that a word must be short enough to start at column j). Then, we will work left to right. For every $j2$, with $j2$ starting at $\mathbf{C} - 1$ and $\text{totals}[i1, j2, i2, j2] = \text{right_column_sums}[j2, 1]$, and for every $j1 < j2$, update $\text{totals}[i1, j1, i2, j2] = \text{totals}[i1, j1 + 1, i2, j2] + \text{right_column_sums}[j1, j2 - j1 + 1]$. The process for words going down is analogous, but with the rows and columns switched, and starting our iteration on columns instead of rows. As $\text{totals}[]$ is finalized for words going both right and down, keep track of the best *fun size* and how many times it occurs. It is also possible to not store the entirety of $\text{totals}[]$ in the Large dataset solution and have a space complexity of $O((\mathbf{R} + \mathbf{C})^3)$ by solving for words going both right and down in the same loop, but doing so requires a slightly different precomputation step.

We still need to analyze the time complexity of the Large dataset solution. Creating $\text{word_lookup}[]$ takes $O(\mathbf{W})$ time. Creating $\text{right}[]$ or $\text{down}[]$ takes $O(\mathbf{R} \times \mathbf{C} \times \mathbf{W})$ time. Creating all $\text{right_column_sums}[]$ takes $O(\mathbf{R}^2\mathbf{C})$ time. Populating $\text{totals}[]$ still has a longer runtime than any of our previously mentioned steps, resulting in our solution taking $O(\mathbf{R}^2\mathbf{C}^2)$ time.

Analysis: Paragliding

Paragliding: Analysis

Small dataset

First, we need a way to determine if a balloon is able to be collected. Note that Edge glides at a 45 degree angle, which means that for each unit he glides horizontally, he also descends one unit vertically. Since Edge can glide down from any position on a tower, Edge can collect any balloon if and only if there is a tower such that the height of the tower is greater than or equal to the horizontal distance between the balloon and the tower. For each balloon i , we can execute a linear search of the towers for a tower j which satisfies $h[j] \geq y[i] + \text{abs}(p[j] - x[i])$. The time complexity for this approach is $O(N \cdot K)$.

Large dataset

We will need to optimize our previous solution in order to have a fast enough solution for the Large dataset. Since we definitely need to iterate through every balloon, we can conclude that we should be optimizing the search for a possible tower for Edge to jump from.

A key observation is that some towers may be irrelevant. For two towers A and B, if the height of A is less than or equal to the height of B minus the distance between A and B, then any balloon which is reachable by A is also reachable by B. A is considered an irrelevant tower. All towers which are not irrelevant can be considered relevant. To eliminate irrelevant towers efficiently, we want to create a sorted array of all towers based on their positions on the x-axis. Then, we want another array to keep track of the relevant towers and their positions. We iterate through the sorted array of all towers and do the following for each tower i : compare tower i to the tower at the end of the array of relevant towers. If tower i is irrelevant with respect to that last tower, continue on to the next tower in the sorted array of all towers; otherwise, continue removing towers from the end of the relevant tower array until both tower i and the last tower are relevant or until the relevant tower array is empty. Then, add tower i to the end of the relevant tower array and continue to the next tower in the sorted array.

The end result is that we have an array of all relevant towers which are also sorted by the towers' positions on the x-axis. Now, for each balloon we can check to see if it is further left or further right than all of the relevant towers. If so, we can check the closest tower to see if it allows Edge to collect the balloon; otherwise if the balloon is not further left or further right, we can use binary search to find the two closest towers to the balloon (one on each side) on the x-axis because we removed all of the irrelevant towers, meaning the only towers which can get to a certain balloon are the closest towers. If Edge can reach the balloon from either of these towers, then the balloon is collectable.

So what is the time complexity of our new solution? Sorting the original list of towers will take $O(N \log N)$ time. During the process of determining relevant towers, each tower cannot be added or removed more than once, so that step is $O(N)$. It is possible that all towers are relevant, so doing a binary search at each balloon will take $O(K \log N)$ time; therefore, our time complexity is $O((N + K) \log N)$.